PRODUCT / ENGINEERING SPECIFICATION

Senior / Principal SWE Trainer

Adaptive interviews, vocabulary fluency, and evidence-based mastery.

Version: 1.0 **Date:** October 20, 2026
Implementation: VS Code with GitHub Copilot or OpenAI Codex
Target devices: Windows desktop, laptop, Android/iOS browser

One codebase. One synchronized learning history. Rigorous questions with clear language.



OVERVIEW

1. Executive brief

Build a responsive, installable Progressive Web App (PWA) that trains an experienced engineer for Senior, Staff, and Principal software-engineering interviews. The platform provides short daily technical sessions, difficult concept questions, a preceding vocabulary primer, text/voice answers, response timers, AI grading, spaced repetition, and longitudinal competency analysis.

> **North star**
>
> **Increase technical challenge as mastery grows, while reducing the burden of unfamiliar jargon.** Measure vocabulary fluency separately from conceptual reasoning. Never use question difficulty as a substitute for actual knowledge.

Decision	Selected approach	Reason
Client	Next.js + React + TypeScript PWA	One web client across desktop, laptop, and phone
State	Supabase Postgres + Auth	Shared, durable, user-isolated history
AI	OpenAI API, server-side	Structured generation, grading, explanations, speech-to-text
Delivery	Vercel or comparable HTTPS host	Simple deployment, mobile access, installability
Local cache	IndexedDB (later)	Draft preservation, future offline mode

Non-negotiable requirements

- **Daily experience:** configurable 5/10/15/20+ questions; Busy, Normal, and Deep modes; topic and difficulty filters.
- **Difficulty:** target about 20% predicted correctness for *new challenge questions*; do not force total session accuracy to 20%.
- **Retention:** a missed question remains in review until **both** 30 days since first miss and 10 consecutive unassisted correct responses.
- **Input:** typed or recorded voice with editable transcript; persistent timer, daily totals, progress, and competency map.
- **Jargon:** 5-10 preread vocabulary cards, inline definitions, a separate terminology mastery track; no reduction in conceptual difficulty.
- **Integrity:** immutable attempt history, versioned questions/rubrics, explicit assist flags, transparent scoring, and honest readiness claims.

2. Scope, user and success measures

Primary audience: One experienced, largely self-taught engineer seeking Senior/Principal performance, with strong practical skills and a desire to systematically learn formal CS terminology. Initial implementation is single-user-friendly but must keep per-user security boundaries.

Primary jobs to be done

- Study for 10-60 minutes from any device and resume without losing progress.
- Answer challenging questions in speech or text while viewing well-formatted code/diagrams.
- Learn unfamiliar terminology without having conceptual ability incorrectly marked as poor.
- See exactly what is improving, what is weak, and what to review next.
- Develop readiness for coding, systems, debugging, system design, and technical leadership.

Success criteria

Measure	Release target
End-to-end viability	Can log in, answer, grade, persist, and resume one question
Durability	No completed attempt or active draft lost on ordinary device switching
Learning loop	Vocabulary primer, question, grade, and spaced review work end to end
Signal quality	First attempt, review, assisted, and vocabulary outcomes separated
Devices	Responsive desktop/laptop/phone plus PWA install where supported

3. Curriculum and competency taxonomy

Every question should map to one primary skill and zero or more secondary skills. Build the taxonomy as a hierarchy rather than a flat tag list. Question and vocabulary IDs must be stable over time.

Domain	Required coverage
C++17 / 20 / 23	RAII, lifetime, value categories, templates, concepts, ranges, coroutines, constexpr, STL, allocators, atomics, memory model
Python 3.10-3.14	Data model, descriptors, decorators, generators, async, typing, interpreter behavior, free-threading, GIL, memory, multiprocessing
Data structures / algorithms	Trees, graphs, tries, hashes, heaps, DP, greedy, sorting, searching, complexity, randomized methods
Concurrency / OS	Thread/process models, scheduling, synchronization, memory ordering, deadlocks, virtual memory, paging, filesystems
Hardware / performance	Caches, NUMA, SIMD, false sharing, profiling, benchmarks, throughput/latency, CPU/GPU interfaces
Networking / data	TCP/IP, DNS, HTTP/TLS, RPC, indexes, transactions, MVCC, LSM, replication
Distributed systems	Consistency, consensus, clocks, failures, sharding, messaging, idempotence, distributed transactions
Engineering & Principal	System design, testing, security, reliability, migrations, architecture trade-offs, incidents, mentoring, cross-team influence

Question categories: Recall, Explain, Compare, Predict, Debug, Design, Critique, Code, Principal Scenario. Emphasize Predict/Debug/Design/Critique for advanced candidates. Track Principal leadership and architecture separately from flashcard knowledge.

4. Session workflow and question UI

```
Dashboard -> Configure -> Vocabulary Primer (5-10 terms)
          -> Question -> Answer/Assist -> Confidence -> Submit
          -> Grade/Explanation -> Next Question
          -> Session Summary -> Competency Dashboard
```

Session settings

Control	Options / behavior
Question count	5 / 10 / 15 / 20 / custom; default 15
Time budget	10 / 20 / 30 / 60 minutes / unlimited
Difficulty	Easy / Normal / Difficult / Brutal / Adaptive; default adaptive
Focus	All skills, a domain/subskill, weak areas, due reviews, Senior, Principal
Preset	Busy: short/review-heavy; Normal: mixed; Deep: long code/system scenarios
Question mix	Indicative 15: 5 due reviews, 2 retention, 5 new hard, 2 weak-skill, 1 Principal wildcard

Question screen information hierarchy

- **Top strip:** current question X/Y, question elapsed time, today total, topic, predicted difficulty; progress bar.
- **Main panel:** prose plus optional syntax-highlighted code, table, KaTeX, thread timeline, Mermaid diagram, graphs or trees.
- **Response:** text box and record/transcribe/edit; show voice permission/fallback status.
- **Actions:** Pause/Resume, Clarify, Look up term, Confidence, Submit; do not auto-advance.
- **After submission:** clear pass/fail and 0-5 rubric, correct reasoning, missing claims, model answer, timing, assists, next question.
- **Cognitive design:** one primary task on screen, stable terms, no distracting animations; competency detail belongs in dashboard.

Question media schema

Use a typed array of safe renderable blocks: **text**, **code**, **table**, **math**, **diagram**. Use an allowlist for syntax and diagram types. Never directly render raw HTML returned by an AI model.

5. Vocabulary and jargon features

FR-VOC-01: Daily primer. Preview the selected daily questions, extract prerequisite jargon, rank unfamiliar terms, then offer 5-10 concise flashcards before the main session. The primer explains wording, not the correct solution to upcoming questions.

FR-VOC-02: Three card types. (1) Term to meaning; (2) meaning to term; (3) distinguish two similar terms. Each term can have formal definition, intuitive explanation, and contextual example.

FR-VOC-03: Inline lookups. Click/tap/select a technical term to reveal a concise definition, optionally add it to the vocabulary deck, and record the assist. A manual "I do not know this term" action must be available even when a term was not pre-annotated.

FR-VOC-04: Independent measures. Terminology mastery and concept mastery are not conflated. A conceptually correct answer after viewing a definition is not automatically wrong, but it is assisted and does **not** count toward unassisted review graduation.

```
Term: synchronizes-with
Formal: A cross-thread relation defined by the C++ memory model.
Intuitive: One mechanism by which another thread safely
           observes earlier effects.
Usage: Explain whether this acquire load synchronizes-with
       the release store and why.
```

6. Answering, timer and grading

Input modes

- **Text:** multiline editor with code-friendly formatting.
- **Voice:** user taps Record/Stop; browser captures audio if permission granted, uploads it securely for transcription.
- **Voice + edit:** show editable transcript before submission; distinguish user-edited transcript from raw transcription.
- **Fallback:** if microphone unavailable, typed answer remains fully usable; never block quiz completion.

Timer specification

Measure question displayed, first interaction, pause intervals, submission, grade, and review intervals. Show question elapsed, active time, daily active total, and question X/Y. Compute duration from timestamps and pause segments; do **not** depend on a browser interval as the source of truth. When devices switch, the latest committed session state wins; protect against duplicate submissions.

Grading schema and rubric

Score	Interpretation
0	Fundamentally incorrect
1	Minimal relevant knowledge
2	Partially correct with major gaps
3	Senior-acceptable under question-specific rubric
4	Strong Senior / approaching Principal reasoning
5	Excellent Principal-level reasoning

Grade against a **question-specific rubric**; default pass is score 3 or more unless the rubric explicitly requires more for a critical concept. Persist score, pass, missing concepts, errors, strengths, ideal answer, assists, grader version, and confidence (Guess / Unsure / Fairly sure / Certain). The grading service must say "uncertain" if a valid score cannot be justified and support re-grade without overwriting the prior result.

> **High-signal mismatch**
>
> "Certain + wrong" should become a high-priority diagnostic item: likely an incorrect mental model. Confidence is an independent signal, not a substitute for correctness.

7. Review and adaptive selection

Failure retention rule (hard requirement)

Once a question is answered incorrectly, create or update its review mastery record. It is eligible to graduate only if **(today >= first_missed_at + 30 days) AND (consecutive_unassisted_correct >= 10)**. Any incorrect attempt resets the consecutive-correct counter. Assisted attempts may contribute to learning analytics but not increase the graduation streak. Early streak completion before day 30 must not prematurely graduate the card.

Use spaced due intervals with randomized jitter, e.g. 1 day, 2-3 days, 4-6 days, 7-10 days, 10-15 days, 15-22 days, 20-30 days. Persist actual due dates; do not assign fixed deterministic review order. Continue scheduling after 10 correct if the 30-day minimum has not yet elapsed.

Difficulty adaptation

Maintain a per-skill ability estimate and a question difficulty estimate. Bootstrap with calibration questions, then estimate P(correct | skill ability, item difficulty). Target approximately **0.20 for new challenge questions only**. Retention questions are deliberately easier if learning is working. Always show both raw accuracy and difficulty-adjusted performance to avoid making improvement look like regression.

For an initial implementation, use a simple logistic or Elo-style model with uncertainty/low-sample indicators. Do not publish percentiles without representative external candidate data. Keep selection explainable: due review, weak domain, new challenge, or wildcard.

Selection constraints

- Deduplicate the same question within a session; avoid recently viewed near-duplicates.
- Select the planned question set *before* producing the vocabulary primer, or re-plan the primer if the set changes.
- Respect time budget and skill preferences without silently dropping overdue reviews.
- If available pool is small, use a transparently labeled fallback rather than fabricate difficulty confidence.
- Store why each question was selected and its estimated P(correct) at selection time.

8. Analytics and readiness

Metric	Definition / special rule
First-attempt accuracy	Unassisted initial answers only; separate from reviews
Review retention	Correctness on scheduled previously-missed items; show 7/30/90 day
Concept mastery	Per skill; incorporate difficulty, rubric, confidence, and uncertainty
Vocabulary mastery	Separate skill/term track; not folded into conceptual scores
Time	Median active seconds overall/by skill, split by question type
Calibration	Actual vs predicted accuracy; highlight confident errors
Learning activity	Today/weekly completed, due, missed, graduated, session minutes
Readiness	Evidence-based label plus underlying assessed domains and sample sizes

Dashboard visualization: A drillable skill-by-level heatmap (Foundation / Senior / Advanced / Principal), plus progress trends and category detail. Show sample sizes, score uncertainty, and raw vs adjusted accuracy. Avoid artificial BigTech candidate percentile estimates.

Readiness labels: Below Senior bar; Approaching Senior; Senior-ready; Strong Senior; Staff-ready; Approaching Principal; Principal-level evidence. Leadership and long-form system design require dedicated assessments; flashcards alone cannot establish Principal readiness.

Example drill-down: Concurrency / C++ Memory Model → relaxed ordering, acquire/release, release sequences, seq-cst, CAS failure ordering, ABA, false sharing, progress guarantees.

9. Cross-device UX and offline states

Desktop

Use spare horizontal space for a larger code/diagram surface and optional collapsible question context. Keep analytics outside the active question by default. Support keyboard shortcuts for pause, submit (with confirmation), next, and term lookup if no typing conflict.

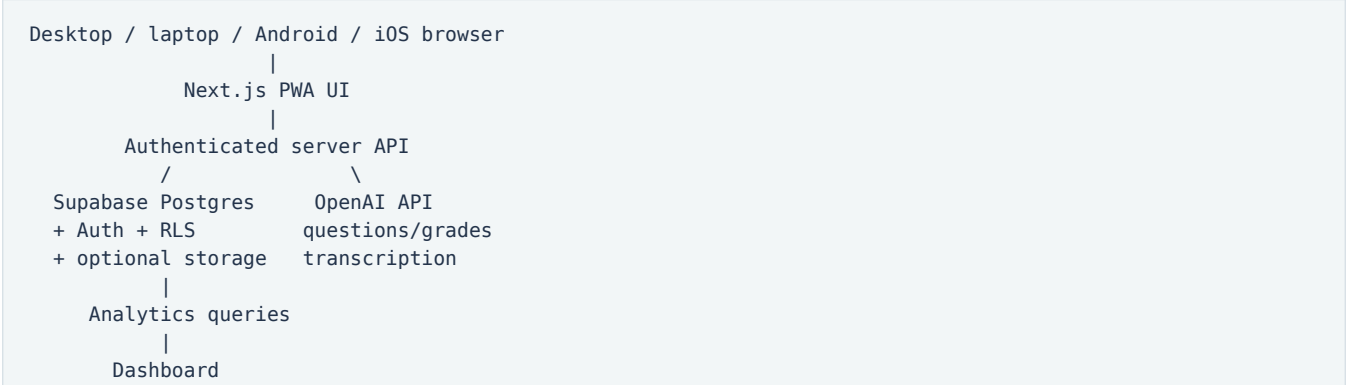
Phone / tablet

Responsive one-column layout, large tappable controls, visible record button, stable text area, minimal navigation during a quiz, and code horizontal scrolling only when necessary. Support both portrait and landscape and system light/dark preferences.

Synchronization and resilience

- Persist attempts and active session state on the server; a user can switch from desktop to phone and resume.
- Store local answer draft snapshots in IndexedDB; restore them on reload or intermittent connectivity.
- Use idempotency keys for submissions, conflict-safe session version checks, and a single active timer authority.
- MVP offline: cached shell + draft; full offline question/attempt queue is a post-MVP feature.
- Never imply recorded voice works offline unless local dictation is available on that device.

10. Architecture and engineering decisions



Concern	MVP decision
Framework	Next.js App Router + TypeScript; React; Tailwind CSS
Backend	Same application: server-only route handlers/services
Identity	Supabase Auth; Google sign-in or passwordless email
Data	Supabase PostgreSQL with strict RLS per user
AI contracts	Versioned prompts and structured JSON outputs with validation
Audio	Browser recording; server-side transcription; delete stored audio by default
Charts	Accessible React heatmap and line/bar chart library
Persistence	Authoritative cloud DB; IndexedDB for drafts
Deployment	HTTPS hosting + app manifest; do not require native installers
Development	VS Code, GitHub repository, package scripts, automated tests

Explicit non-goals for MVP

No Kubernetes, microservices, custom queue infrastructure, real-time collaboration, native mobile builds, candidate percentile claims, automated competitive leaderboards, or full offline voice transcription. SQLite can be used for isolated tests but is **not** the canonical user history.

11. Relational data model

Use migrations checked into the repo. All user-owned tables include user_id with authorization enforced by RLS. History is append-only wherever feasible. Questions and grading rubrics must be versioned rather than mutated in place.

Entity	Minimum fields / relationship
profiles	user_id PK, created_at, timezone, preferences
curriculum_skills	id, parent_id FK, domain, skill_name, level
questions	id, canonical_skill_id, status, created_at
question_versions	id, question_id FK, version, type, content_blocks JSONB, rubric JSONB, difficulty, model/prompt_version
question_skills	question_version_id FK, skill_id FK, relation_weight
vocabulary_terms	id, canonical_term, formal_definition, intuition, example
question_terms	question_version_id FK, term_id FK, required, spoiler_risk
sessions	id, user_id FK, state, preset, planned_count, started_at, completed_at, config JSONB, version
session_items	id, session_id FK, question_version_id FK, sequence, source, selected_probability, state
attempts	id, user_id FK, session_item_id FK, question_version_id FK, answer_text, input_mode, confidence, active_ms, submitted_at, assisted, idempotency_key
attempt_grades	id, attempt_id FK, rubric_version, grader_version, score, passed, details JSONB, created_at
question_mastery	user_id FK, question_id FK, first_missed_at, streak, next_due_at, graduated_at, counters
vocabulary_mastery	user_id FK, term_id FK, streak, next_due_at, skill_estimate, attempts
ability_estimates	user_id FK, skill_id FK, theta, uncertainty, last_updated
ability_snapshots	user_id FK, skill_id FK, measured_at, theta, sample_size

History invariants

- Attempts immutable after submission; an updated grade appends a new grading revision and marks which grade is current.
- Unique (session_id, sequence) and unique user-scoped idempotency key on submit.
- Persist question_version_id + rubric version + grader version for reproducibility.
- Transactionally store an attempt, grade reference, mastery update, and session progress; retry safely.
- Use UTC timestamps in database; render local timezone in UI.

12. API contracts (initial)

Endpoints are a proposed server API shape, not a restriction on Next.js route-handler organization. Protect all user-scoped routes with session identity. Validate with Zod or equivalent, apply limits, and return typed error codes.

Method / endpoint	Purpose
GET /api/me	Profile, timezone, active session, study defaults
GET /api/dashboard	Today counts, skill summaries, due items, time totals
POST /api/sessions	Create session from config; plan items and primer

Method / endpoint	Purpose
GET /api/sessions/:id	Resume current state and completed item list
POST /api/sessions/:id/timing	Pause/resume or checkpoint active timing
POST /api/questions/:id/terms	Resolve selected jargon and record assist
POST /api/answers/transcribe	Upload short recording, return transcript
POST /api/attempts	Idempotent answer submission; return attempt ID
POST /api/attempts/:id/grade	Generate and persist structured grade
GET /api/reviews/due	Fetch scheduled concept + vocabulary reviews
GET /api/competencies	Heatmap/drill-down data and sample sizes
GET /api/history	Filterable attempt/activity timeline

Representative request/response

```
POST /api/attempts
{
  "sessionItemId": "uuid", "questionVersionId": "uuid",
  "answerText": "...", "inputMode": "voice_edited",
  "confidence": "certain", "activeDurationMs": 103000,
  "termIdsRevealed": ["uuid"], "hintUsed": false,
  "idempotencyKey": "client-generated-uuid"
}

201 Created
{ "attemptId": "uuid", "gradingState": "pending" }
```

For long AI operations, persist a pending grading state and expose polling or a supported streaming mechanism. A failed grading call must not lose the submitted answer.

13. AI content and grading contracts

Question object

```
{
  "questionId": "uuid", "version": 1,
  "primarySkillId": "cpp.memory_order",
  "type": "predict", "difficulty": 4.6,
  "contentBlocks": [{"type":"text","markdown":"..."},
                    {"type":"code","language":"cpp","source":"..."}],
  "terms": ["release-store", "relaxed-load"],
  "rubric": [{"claim":"...", "points":2}],
  "referenceAnswer": "...", "promptVersion": "qgen-v1"
}
```

Grade object

```
{
  "score": 3, "passed": true,
  "correctClaims": ["..."], "missingClaims": ["..."],
  "incorrectClaims": [], "idealAnswer": "...",
  "confidenceMismatch": "confident_wrong",
  "conceptTags": ["cpp.memory_order"],
  "needsManualReview": false,
  "graderVersion": "grader-v1"
}
```

The production grading schema should enforce question-specific pass criteria, supported confidence-mismatch values, 0-5 integer score, and explanatory evidence. Use deterministic validators for outputs, safe retries, and a set of human-reviewed "golden" attempts to detect grading drift.

AI design rules

- Do not expose answer rubrics/reference answers to the client until grading is complete.
- Store complete question/rendered version at first exposure; avoid silently regenerating while a question is active.
- Constrain explanations to the rubric and source code; flag ambiguous or flawed questions for revision.
- Treat user answers and third-party question text as untrusted input, not instructions to backend tools.
- Use a less costly model for straightforward terminology/tasks; use stronger reasoning for complex grading where validated.

14. Algorithms and state transitions

Missed-question state machine

```
NEW --wrong--> REVIEW_ACTIVE
REVIEW_ACTIVE --unassisted correct--> REVIEW_ACTIVE (streak++)
REVIEW_ACTIVE --wrong--> REVIEW_ACTIVE (streak=0)
REVIEW_ACTIVE --assisted correct--> REVIEW_ACTIVE (streak unchanged)
REVIEW_ACTIVE --both requirements fulfilled--> GRADUATED
GRADUATED --retention failure--> REVIEW_ACTIVE (new streak=0)
```

Mastery graduation pseudocode

```
onGrade(question, attempt, grade):
  if grade.passed == false:
    if first_missed_at is null: first_missed_at = now
    consecutive_correct = 0
  else if not attempt.assisted and grade.passed:
    consecutive_correct += 1

  eligible = (now >= first_missed_at + 30 days)
             AND (consecutive_correct >= 10)
  if eligible: graduated_at = now
  else: next_due_at = spacedDueDate(streak, jitter)

  persist atomically; never graduate on elapsed time alone
```

Daily planning pseudocode

```
planSession(config, skillState, dueQueue, questionBank):
  slots = chooseMix(config.preset, config.questionCount,
                    dueQueue, config.timeBudget)
  for slot in slots:
    if slot == 'new':
      sample difficulty near P(correct)=0.20
    if slot == 'review':
      sample eligible due item with randomized ordering
    avoid duplicates and near-duplicates
  primer = selectUnknownPrerequisiteTerms(plannedQuestions)
  save plan + primer before starting the session
```

Implementation detail: the consecutive correct count concerns graded, unassisted **review responses** to the same canonical question. Acceptable success is defined by that question version/rubric. Version upgrades that materially change the concept should start a new mastery lineage or explicitly migrate state.

15. Security, accessibility and reliability

Security / privacy

- OpenAI credentials and privileged Supabase keys remain server-only; never embed in client bundles.
- Enforce per-user RLS on profile, sessions, attempts, mastery, preferences, and derived analytics.
- Rate-limit AI-generation/transcription routes and validate maximum input/audio sizes.
- Audio is deleted after transcription by default; disclose retention policy; never log complete audio or secrets.
- Protect against prompt injection from user answers/question content and unsafe Markdown/diagram rendering.
- Supply export/delete-account pathways before broad public launch.

Accessibility / cognitive design

- Keyboard navigation, readable focus indicators, text alternatives for diagrams, meaningful form labels, sufficient contrast.
- Do not use time pressure to force automatic submission; allow pause and session resume.
- Screen-reader announcements for timer state and grading result; avoid announcing every timer tick.
- Define jargon concisely with formal + practical explanations without spoiling problem answers.
- Distinguish "term unknown" from "concept unknown" in UI, grading, and analytics.

Observability

Capture privacy-safe events for session start/resume, answer submit, grade success/failure, transcript errors, AI latency, and sync conflict. Use request correlation IDs. Retain prompt/model versions and cost/latency telemetry; never log audio content or sensitive access tokens.

16. Release plan and test strategy

Phase	Includes	Exit gate
MVP 1: Core loop	Auth, Postgres, curriculum, formatted question, timer, text input, grade, history	One answer round-trip works and survives reload/device switch
MVP 2: Learning	Vocabulary primer, lookups, review queue, question selection, confidence, sessions	15-question adaptive session, accurate review state
MVP 3: Interface	Voice/edit, diagrams, responsive PWA, competency heatmap, trend charts	Usable on Windows/laptop/Android/iPhone; installable where supported
V2: Principal depth	Long coding, architecture critique, system design, leadership simulations	Rubric-based deep exercise completion
V2: Calibration	Golden evaluation set, model comparison, difficulty calibration, uncertainty	Readiness claims tied to validated evidence

Automated tests (minimum)

- **Unit:** review graduation requires both 30 days and 10 consecutive unassisted correct; wrong resets streak; assisted correct does not increment.
- **Unit:** first-attempt accuracy excludes repeats and assisted first tries; vocabulary and concept metrics are distinct.
- **Unit:** timer excludes pause segments and aggregates across sessions for user-local day.
- **Integration:** duplicate POST /attempts is idempotent; grading retry does not duplicate attempts.
- **Integration:** user A cannot read or mutate user B records via API or DB.
- **E2E:** log in, configure, primer, answer, assistance, confidence, submit, grade, next, summary.
- **E2E:** answer started on desktop is resumed on mobile width after refresh.
- **Accessibility:** keyboard-only flow, focus visibility, readable code/diagram fallbacks and mobile tap targets.

17. MVP acceptance checklist

[ ] **AC-01** Authenticate and access the same account from desktop, laptop, and phone.

[ ] **AC-02** Configure question count, time budget, target difficulty, focus and study preset.

[ ] **AC-03** Run a relevant 5-10-card vocabulary primer without leaking main question solutions.

[ ] **AC-04** View well-formatted technical questions with code, tables and diagrams where appropriate.

[ ] **AC-05** Answer using text or recorded speech with editable transcript.

[ ] **AC-06** See accurate active question timer, paused time, total today, and X/Y session progress.

[ ] **AC-07** Request inline definition for unfamiliar jargon and flag assistance.

[ ] **AC-08** Record confidence before answer feedback is revealed.

[ ] **AC-09** Submit and receive structured 0-5 rubric grade, pass flag and actionable explanation.

[ ] **AC-10** Persist immutable attempt metadata including exact question/rubric versions.

[ ] **AC-11** Apply both failed-question retention requirements and due scheduling correctly.

[ ] **AC-12** Select challenge questions adaptively without confusing review performance with first attempts.

[ ] **AC-13** Show skill competency heatmap and distinct vocabulary/knowledge/fluency metrics.

[ ] **AC-14** Resume interrupted session from another device without duplicate answers.

[ ] **AC-15** Install as PWA where supported; keep basic draft recoverability when connection fails.

18. Repository skeleton

```
swe-trainer/
  app/
    (auth)/                # login / callback
    (app)/                 # dashboard, study, progress, settings
    api/                   # typed server handlers
  components/
    question/  vocabulary/  timer/  analytics/
  lib/
    ai/          # prompt templates, schema, graders
    db/          # typed queries and migrations
    scheduling/  # spaced repetition, adaptive planner
    analytics/   # aggregate metrics
    validation/  # shared Zod schemas
  supabase/migrations/
  tests/
    unit/  integration/  e2e/  fixtures/
  docs/
    decisions/  prompts/  api-contracts/
  public/
    manifest.webmanifest  icons/
  .env.example
  README.md
```

Recommended developer commands: `npm run dev`, `npm run lint`, `npm run typecheck`, `npm test`, `npm run test:e2e`. Provide .env.example and concise local setup steps. Keep secrets out of version control. A Supabase local development environment may be containerized with a Docker/Podman-compatible workflow where practical.

19. Agent implementation brief

The following instruction is intended to be copied into a fresh VS Code Codex/Copilot agent session together with this PDF. It keeps the implementation incremental and testable.

```
Implement the attached "Senior / Principal SWE Trainer" spec.
Use Next.js App Router + TypeScript + Supabase Postgres/Auth,
Tailwind, and server-side OpenAI calls. Build a responsive PWA.

Start with Phase 1 only: authentication, migrations, seeded
curriculum + sample questions, question UI, timing, text answer,
structured grading, immutable attempt history, and resume.

Requirements:
1. First inspect repo; make a concise implementation plan.
2. Set up typed contracts, DB migrations, RLS and tests first.
3. Never put API keys in client code.
4. Implement the vertical slice and run lint/typecheck/tests.
5. Write README with setup, .env.example, and what works.
6. Do not claim Phase 2/3 features are complete yet.
7. Record decisions and outstanding spec questions in docs/.

Important invariants for future phases:
- Missed-item graduation requires 30 days AND 10 consecutive
  unassisted correct responses.
- Separate vocabulary, first-attempt, and review metrics.
- ~20% predicted success applies to NEW challenge items only.
- Persist exact question/rubric/grader versions and timing.
- Cloud DB is authoritative; support cross-device sessions.
```

Implementation sequencing

First vertical slice: sign in -> fetch seeded question -> start timing -> type answer -> submit idempotently -> grade -> persist attempt -> reload and display same completed attempt. This should be demonstrated before implementing all adaptive logic or charts.

> **Definition of done for an agent-authored change**
>
> A feature is complete only when it has migrations (if needed), typed schema, server validation, meaningful UI states, error handling, automated tests, and documentation. Do not count a mockup as implemented functionality.

20. Assumptions and decisions to revisit

- **Costs:** Model usage is billed separately from ChatGPT subscriptions; measure generated tokens and transcription usage rather than guessing monthly cost.
- **One-person-first:** Data model supports multiple users, but no social or classroom features in MVP.
- **Question correctness:** AI-generated content can be flawed; design a flag/report/revision loop and maintain golden question fixtures.
- **Scoring:** Initial readiness labels are criterion-referenced, not externally validated percentiles.
- **Time:** Use UTC storage semantics with UTC DB timestamps; local day follows the user profile timezone, defaulting to the browser-detected zone.
- **Mobile audio:** Browser permission and audio format support differ by device; ensure manual text fallback.
- **Scope:** Full offline sessions and proctored/benchmark exams can come later.

Document history: v1.0 consolidates the agreed features and adds implementation-oriented requirements, persistence constraints, API contracts, and verification gates. It is a build specification, not a claim that the application already exists.